People of the Holocaust

System Design Models

Marquette University
COSC 4920 – Principles of Design

Version 1.0
October 19, 2026

Contents

1 Introduction **2**
1.1 Purpose . . . 2
1.2 Scope . . . 2
1.3 Model Summary . . . 2

2 UML Use Case Diagram **3**

3 UML Activity Diagram **4**

4 UML Sequence Diagram **6**

5 UML State Diagram **7**

6 UML Class Diagrams **9**
6.1 Domain Model . . . 9
6.2 Front End . . . 11

7 Block Diagram **12**

8 Database Schema **13**

List of Figures

1 Use case diagram for the public website . . . 3
2 System context activity diagram . . . 5
3 Sequence diagram for updating the database and user interaction . . . 6
4 System state diagram . . . 8
5 Domain class diagram . . . 10
6 Front end class diagram . . . 11
7 System architecture block diagram . . . 12
8 Database schema . . . 13

1 Introduction

1.1 Purpose

This document collects the design models for the *People of the Holocaust* senior design project. Each model describes one view of the system: how visitors use the website, how work flows through the interpreter and the website, how the system changes state, how the parts communicate over time, how the code is structured, and how the major components connect. Each diagram is followed by a short description of what it shows.

1.2 Scope

The models cover the three main parts of the system: the Python interpreter that processes text from the *Encyclopedia of Camps and Ghettos* (ECG), the project database that stores the structured records, and the React and TypeScript website that presents those records to the public. The database schema is included as a supplementary model.

1.3 Model Summary

Section	Model	Status
2	UML Use Case Diagram	Required
3	UML Activity Diagram	Required
4	UML Sequence Diagram	Required
5	UML State Diagram	Required
6	UML Class Diagrams (domain and front end)	Required
7	Block Diagram (architecture deliverable)	Required
8	Database Schema	Supplementary

Table 1: Design models included in this document

2 UML Use Case Diagram

The use case diagram shows what a visitor can do on the public website. A single actor, the website visitor, is connected to nine use cases. No account or login is involved, so every use case is available to any visitor.

- **Project information:** read the project background.
- **People:** browse people, search people by name, and view a person's details.
- **Places:** browse places, search places by name, and view a place's details.
- **Map:** explore the historical map and view a place's information by selecting its marker.

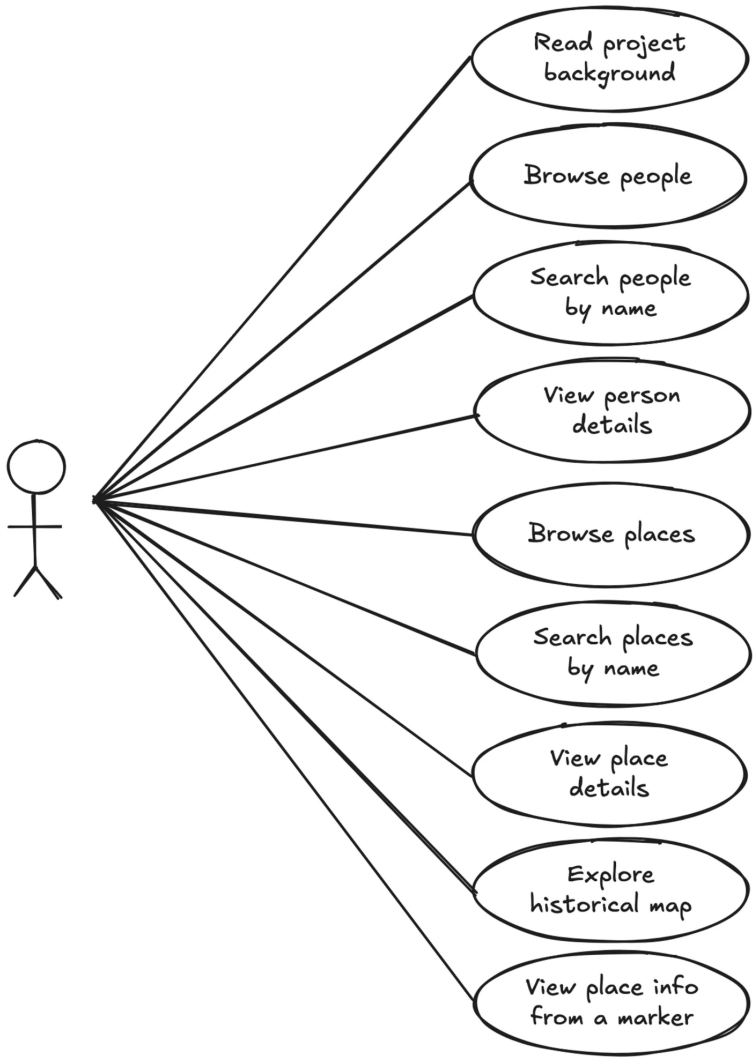


Figure 1: Use case diagram for the public website

3 UML Activity Diagram

The activity diagram, titled *System Context Activity Diagram*, follows two independent flows across four swimlanes: *Project Maintainer*, *PotH System*, *Source Environment*, and *Public Visitor*. A fork at the start shows that the two flows run separately; as the attached note states, website access does not depend on an interpreter run completing.

- **Processing flow.** The maintainer initiates source processing. The interpreter prepares structured person-name records, then loops while source articles remain. For each article, the ECG provides the source text and the interpreter retrieves and cleans it. On success, the interpreter identifies sentences that contain known people, associates activities with people and places, and the database stores the processed records with their source context. On failure, the error is logged for later review. When no articles remain, the maintainer reviews the processing results and error logs.
- **Visitor flow.** The visitor accesses the public website, which presents the project background and navigation without a login. The visitor chooses what to explore: project background is displayed directly, while historical records are supplied by the database and presented as records or map locations. Selecting a person, place, or map marker displays its related details and context. The visitor continues browsing until leaving the website, and both flows join at the end.

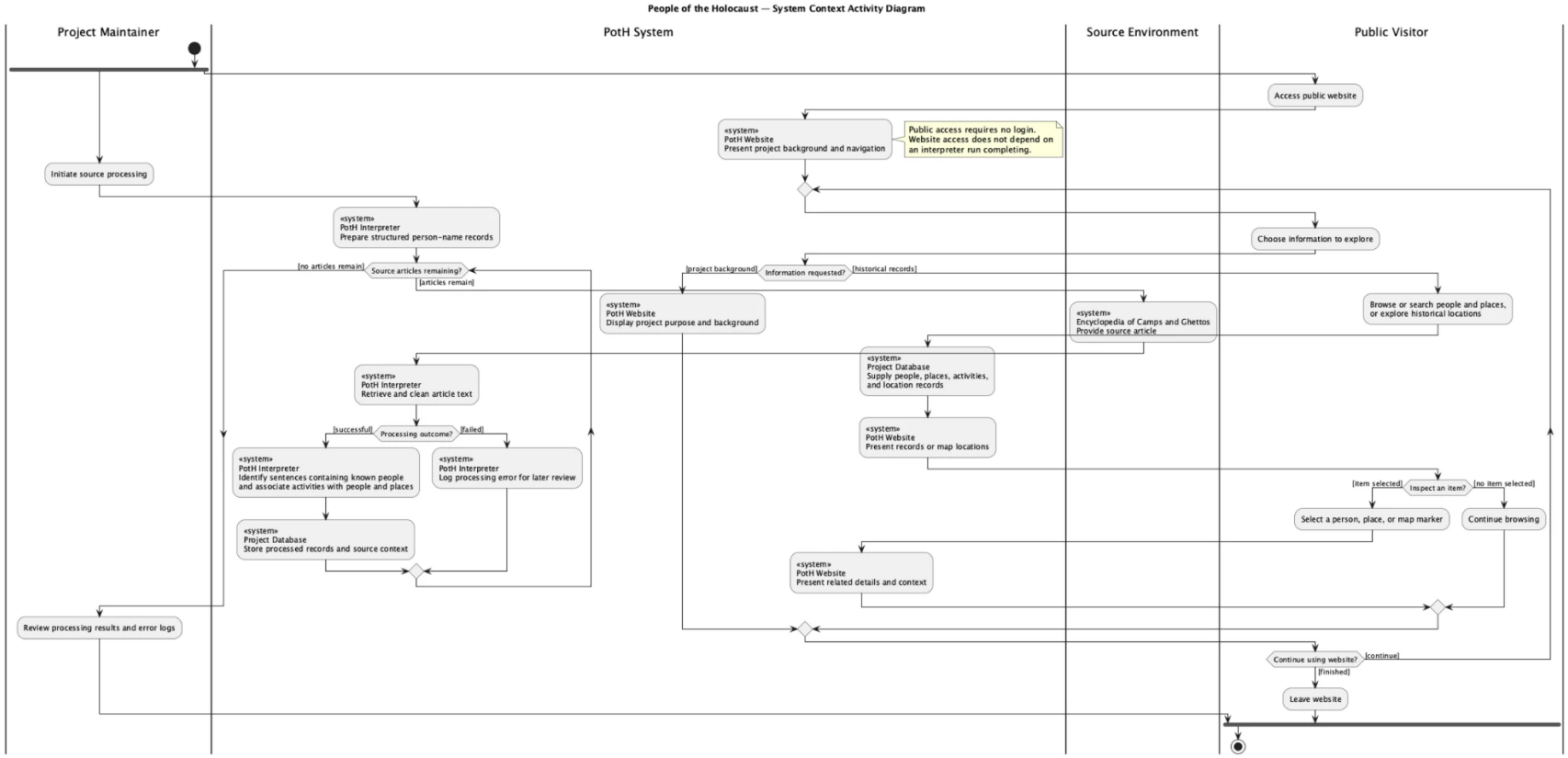


Figure 2: System context activity diagram

4 UML Sequence Diagram

The sequence diagram shows the messages exchanged between six participants over time: the NHM Database (the encyclopedia source), the Web Scraper, the NLP component, the POTH Database, the Frontend, and the User. It is divided into two frames.

- **Updating Database.** The Web Scraper requests the encyclopedias from the source and receives them, stores the response in the POTH Database, and receives confirmation. It then passes the entries to the NLP component, which stores the resulting NLP tree data in the POTH Database and receives confirmation. The Web Scraper and NLP lifelines end once their work is complete.
- **User Interaction.** The User opens the homepage and the Frontend displays it. When the User submits a query, the Frontend requests the necessary information from the POTH Database, receives it, and formats and displays it to the User. The Frontend lifeline ends when the User closes the session.

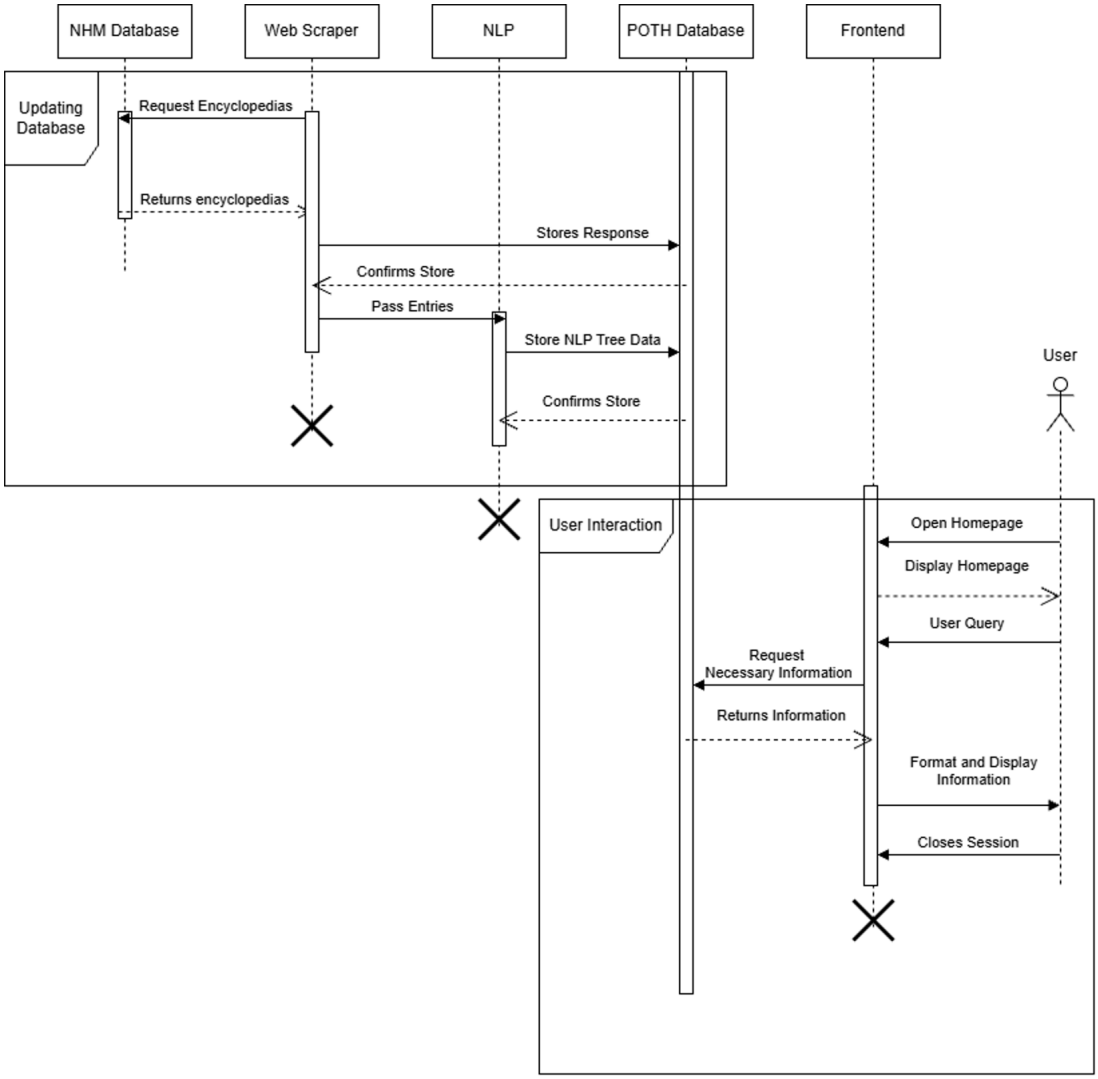


Figure 3: Sequence diagram for updating the database and user interaction

5 UML State Diagram

The state diagram, titled *System State Diagram*, models the system as a single *Operational* state containing two concurrent regions: *Processing* and *Website*.

- **Processing.** The interpreter waits in *AwaitingRun* until the maintainer starts a run, then moves to *Preparing*. If preparation fails, it enters *Failed* and returns to *AwaitingRun* once the issue is resolved. Otherwise it enters *Checking*: while articles remain it cycles through *Reading*, *Extracting*, and *Storing*, returning to *Checking* after each article. A failure while reading, extracting, or storing moves to *Logging*, and processing continues with the next article once the error is recorded. When no articles remain, the run is *Complete* and returns to *AwaitingRun* when acknowledged.
- **Website.** The website is *Available* until a visitor opens it, starting a *Session* that begins at *Home*. From any page the visitor can navigate to *People*, *Map*, or *Places*. The People and Places states each move between a filtered list and a details view, and the Map state moves between *Exploring* and *MarkerInfo* when a marker is selected or its popup is closed. The session ends when the visitor leaves.

The notes record that article failures are logged without stopping processing, that database records persist between runs, that each session represents one visitor with no authentication, and that missing optional fields do not prevent details from being viewed.

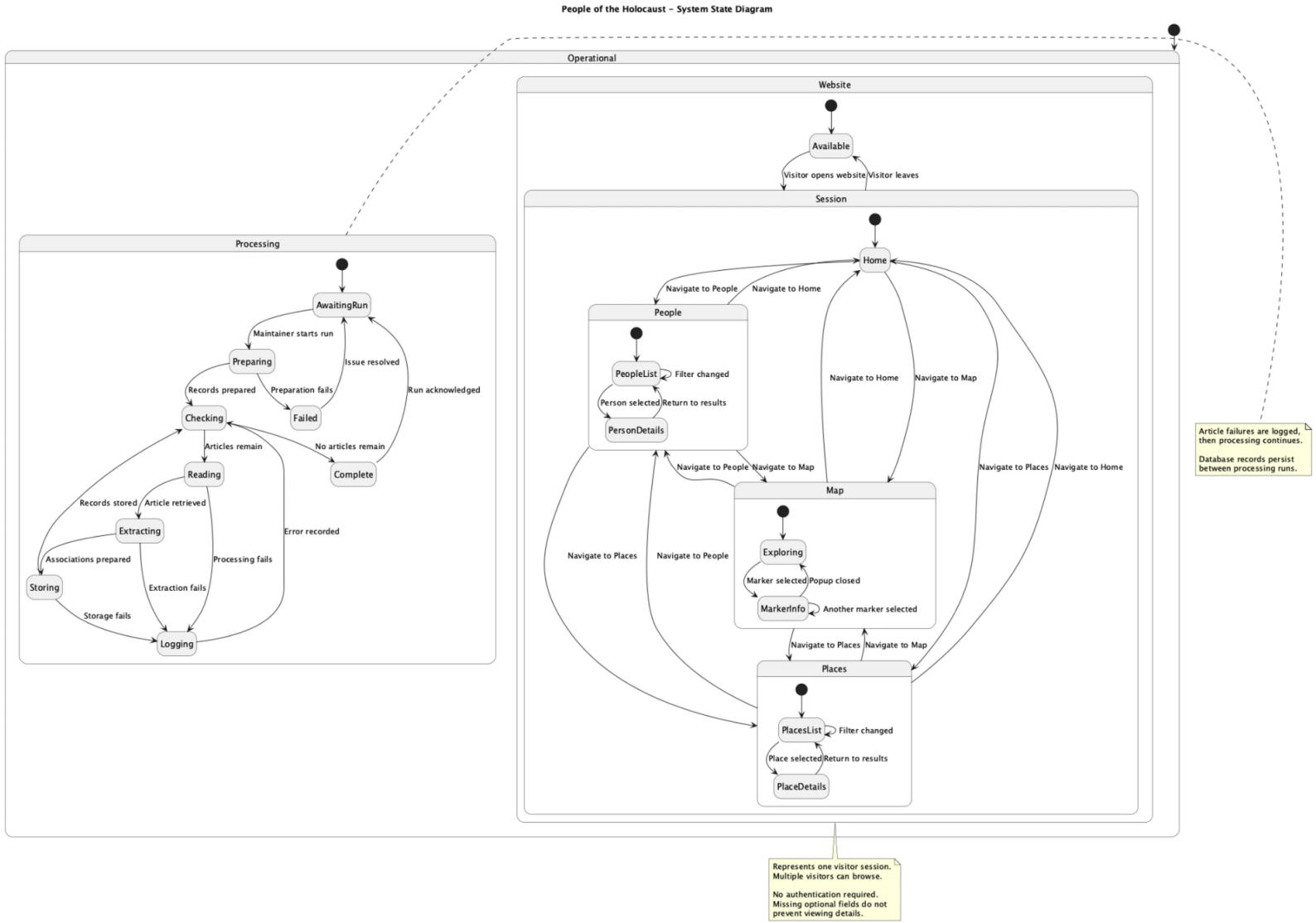


Figure 4: System state diagram

6 UML Class Diagrams

Two class diagrams describe the structure of the system: a domain model of the data the system stores, and a model of the front end website.

6.1 Domain Model

The domain model shows the encyclopedia's structure and the people and places drawn from it.

- **Text hierarchy.** A `Volume` contains many `Article`s, and each `Article` contains many `Paragraph`s (composition). Each paragraph records its position and its start and end pages, so `citation()` can give the volume and page for any quoted text.
- **Mentions.** A `Paragraph` mentions many `Person`s and many `Place`s, and each person or place can be mentioned in many paragraphs (many-to-many associations).
- **Places.** An `Article` is about at most one `Place`, which lets each place lead with its own article.
- **People.** Each `Person` is listed under one or more `NameEntry` records, so alternate spellings of a name refer to the same person. A person's kind is given by the `PersonKind` enumeration: Individual, Family, or Brother.

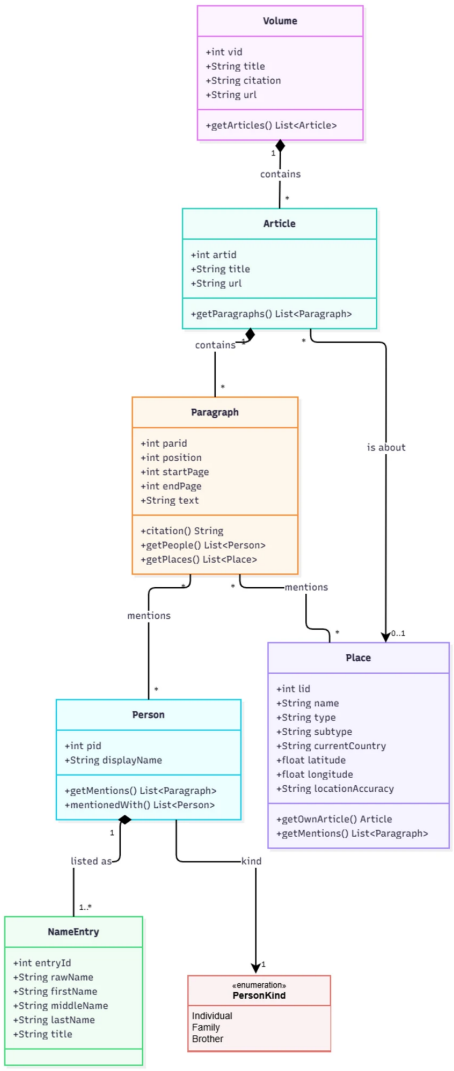


Figure 5: Domain class diagram

6.2 Front End

The front end class diagram shows the React components of the website and the data they display.

- **Website and pages.** `Website` holds the navigation shown on every page and is composed of three pages. `HomePage` presents the project and loads no data from the database.
- **People page.** `PeoplePage` combines a `SearchBar`, a `PeopleList` of results, and a `PersonDetails` panel shown once a person is selected.
- **Places page.** `PlacesPage` combines the same `SearchBar`, an interactive `MapView`, a `PlaceList`, and a `PlaceDetails` panel. Selecting a place from either the map or the list opens its details.
- **Quotes and citations.** Both details panels display a list of `QuoteCard`s. Each card shows one paragraph from the encyclopedia with the name highlighted and its citation.
- **Data.** Only `DataLoader` retrieves data. The `<<type>>` classes describe the shape of the data it returns, with `Quote` holding a paragraph's text, volume, article, and pages.

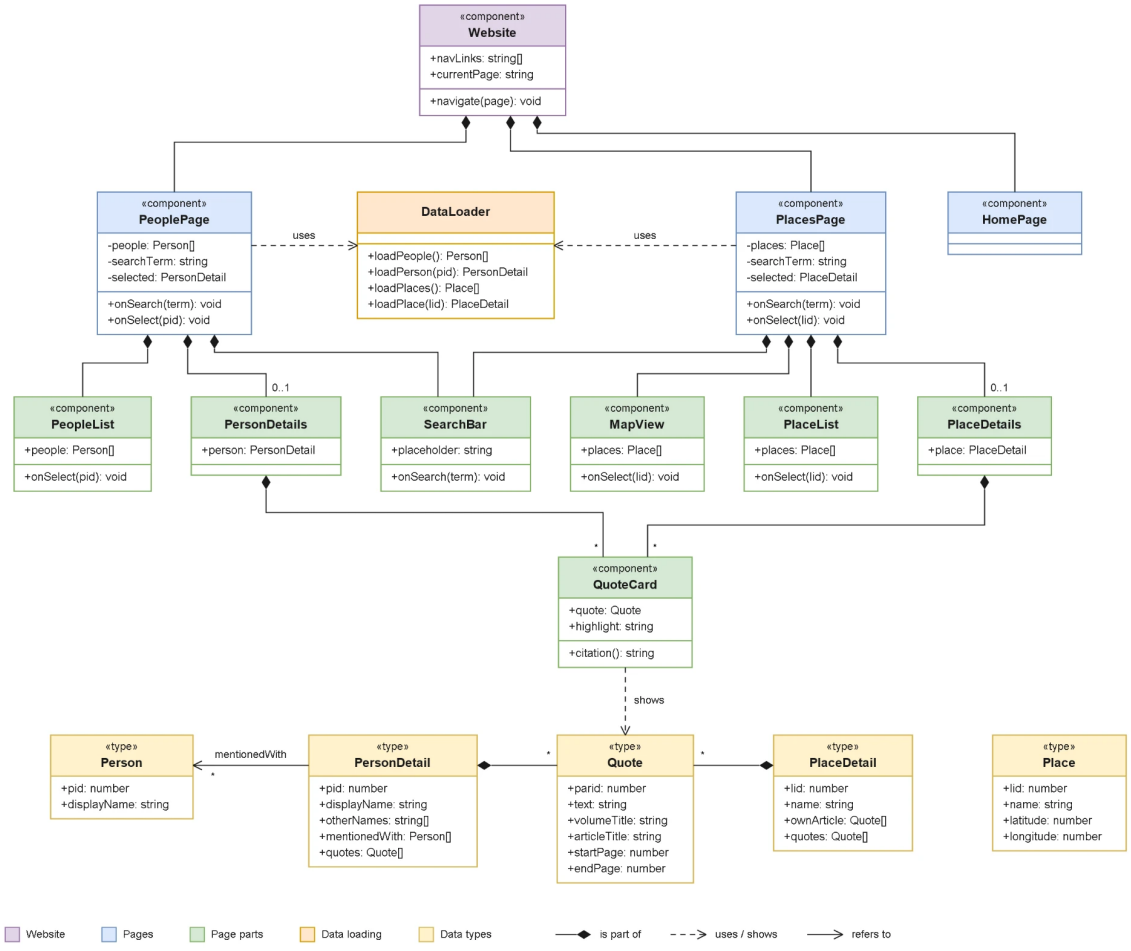


Figure 6: Front end class diagram

7 Block Diagram

The block diagram is the architecture deliverable. It shows the four major components of the system and the data passed between them.

- The **Web Scraper** saves the encyclopedia to the **Database** and passes it to the **NLP** component as input.
- The **NLP** component processes the text and saves the resulting relational tree to the **Database**.
- The **Website** requests relational data from the **Database**, which fulfills those requests.

The Database is the only component that both the processing side and the website use, which keeps the website independent of when processing runs.

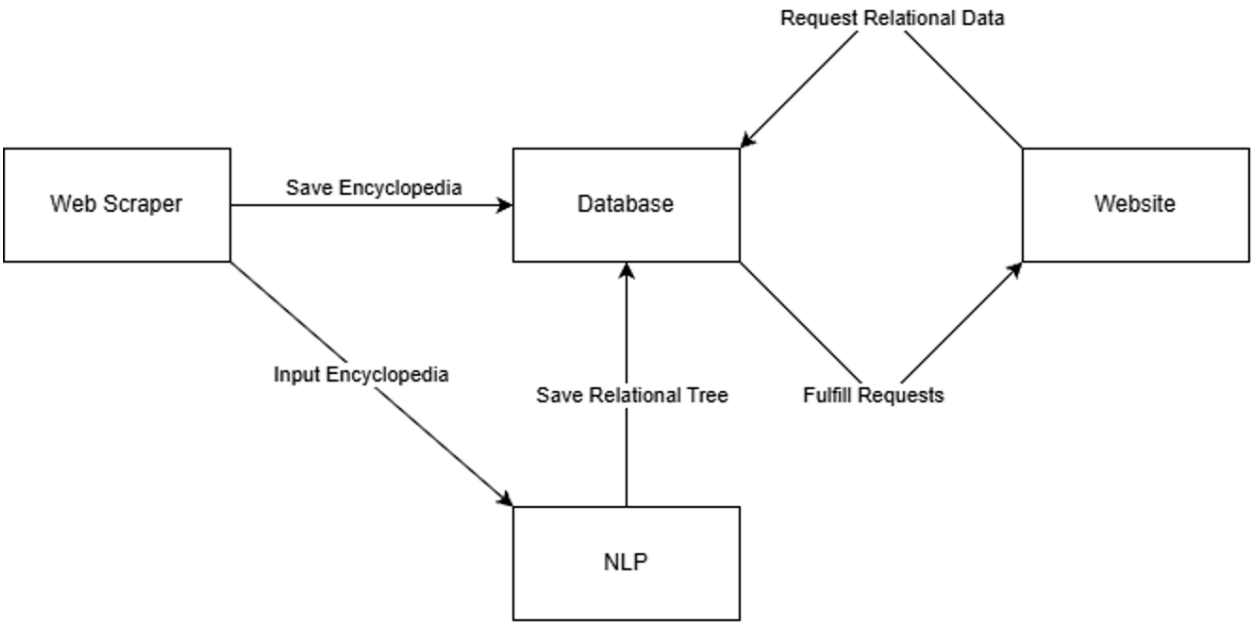


Figure 7: System architecture block diagram

8 Database Schema

The schema is a supplementary model showing how the project database stores the records described by the domain model. Tables are grouped into three areas.

- **Encyclopedia tables.** `Volume`, `Article`, and `Paragraph` store the source text. Each paragraph stores its position within its article and its start and end pages, and a unique index on article and position keeps paragraphs in order.
- **Places tables.** `Places` stores each location with its type, country, and coordinates. `Where_Place_Mentioned` links places to the paragraphs that mention them.
- **People tables.** `People` stores each person, and `Name_Entry` stores every name under which a person is listed. `Where_Person_Mentioned` links people to the paragraphs that mention them.

Both mention tables use the paragraph and the person or place as a composite primary key, so each person or place is recorded at most once per paragraph. Additional indexes support searching by name and building each person's and place's page.

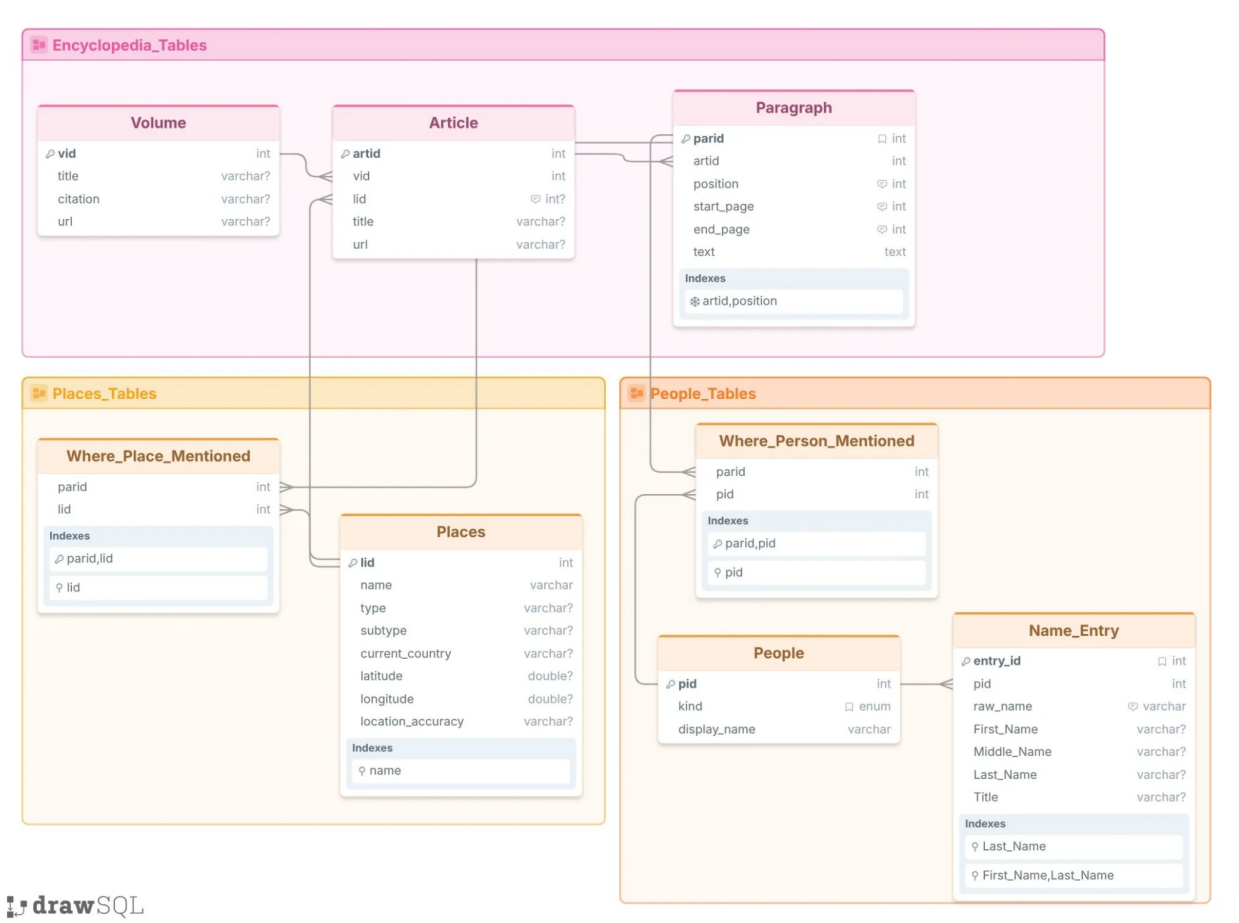


Figure 8: Database schema